STL – SEQUENCE CONTAINERS

CARACTERISTICAS

- Proporciona la implementación de estructuras de datos comúnmente usadas, por ejemplo: listas enlazadas y colas.
- Las estructuras de datos son implementadas usando contenedores, los cuales almacenan datos llamados elementos.
- Son colecciones homogéneas, es decir, se puede crear un contenedor para un solo tipo de dato.
- Es importante familiarizarse con las estructuras de datos disponibles para elegir la más apropiada para una tarea.

ALGUNOS CONTENEDORES DISPONIBLES

Sequence containers

Sequence containers implement data structures which can be accessed sequentially.

array (C++11)	fixed-sized inplace contiguous array (class template)
vector	dynamic contiguous array (class template)
inplace_vector (C++26)	dynamically-resizable, fixed capacity, inplace contiguous array (class template)
deque	double-ended queue (class template)
forward_list (C++11)	singly-linked list (class template)
list	doubly-linked list (class template)

Associative containers

Associative containers implement sorted data structures that can be quickly searched (*O(log n)* complexity).

set	collection of unique keys, sorted by keys (class template)
map	collection of key-value pairs, sorted by keys, keys are unique (class template)
multiset	collection of keys, sorted by keys (class template)
multimap	collection of key-value pairs, sorted by keys (class template)

Unordered associative containers (since C++11)

Unordered associative containers implement unsorted (hashed) data structures that can be quickly searched (*O(1)* average, *O(n)* worst-case complexity).

unordered_set (C++11)	collection of unique keys, hashed by keys (class template)
unordered_map (C++11)	collection of key-value pairs, hashed by keys, keys are unique (class template)
unordered_multiset (C++11)	collection of keys, hashed by keys (class template)
unordered_multimap (C++11)	collection of key-value pairs, hashed by keys (class template)

ARRAY

Es un reemplazo de los arreglos estándar de C, análogos en estructura y funcionamiento.

- Se debe usar cuando se tienen colecciones de un tamaño constante.

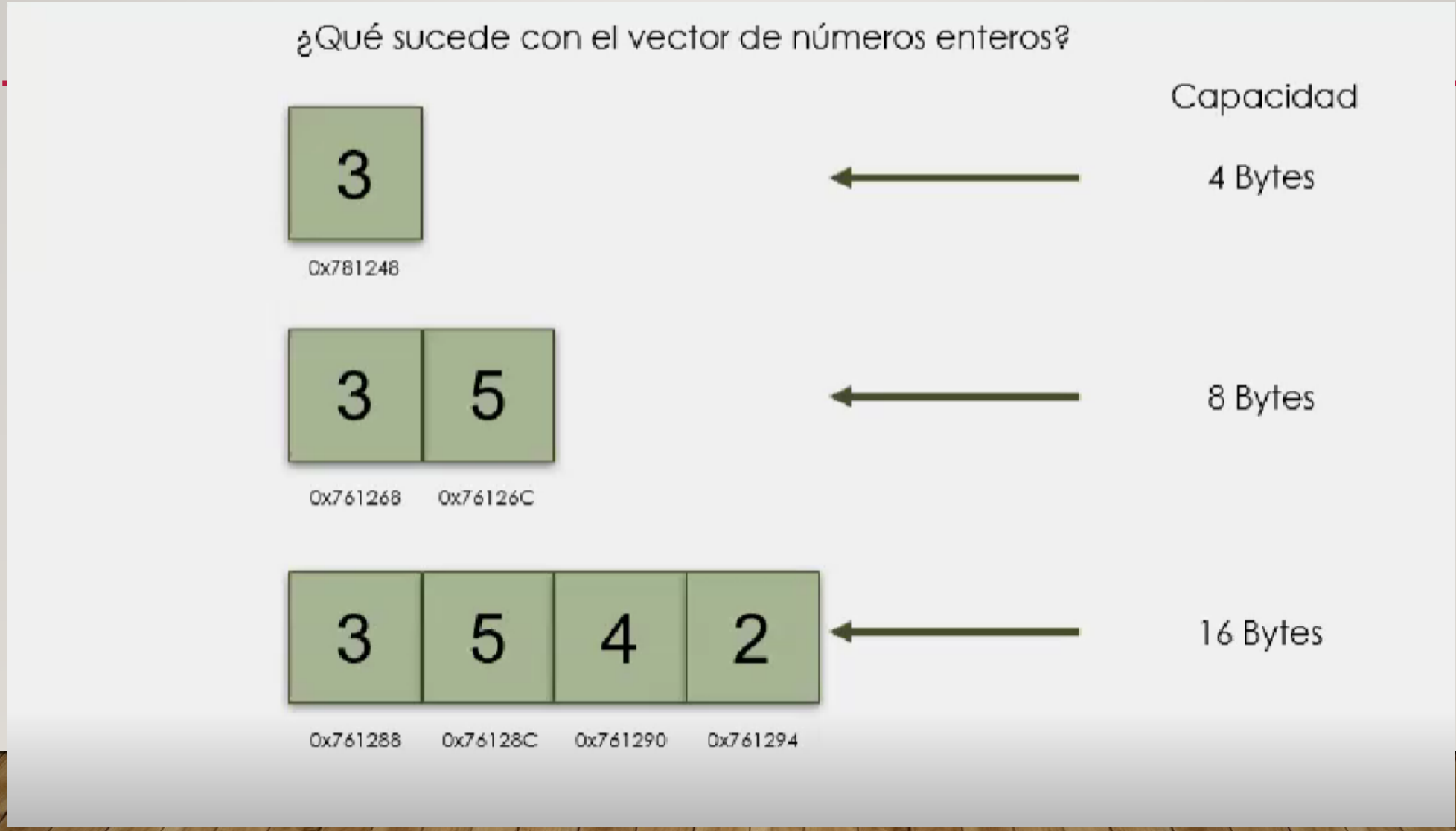
- Las ventajas principales respecto de los arreglos convencionales, es que siempre conoce su propio tamaño y que se tiene un acceso muy rápido a los elementos así como con el contenedor vector.

VECTOR

Almacena una secuencia, de elementos y permite acceso a ellos a través de sus diferentes métodos.

- Análogo en estructura a un arreglo de elementos, pero puede crecer dinámicamente.
- Al igual que un arreglo, los elementos del vector se almacenan contiguos en memoria.
- Se debe usar vectores cuando se necesita rápido acceso a los elementos, pero se debe evitar añadir o remover elementos entre el arreglo.

MANEJO DEL ALMACENAMIENTO CLASE VECTOR



std::vector

Defined in header <vector>

```
template<
    class T,
    class Allocator = std::allocator<T>
> class vector;
```
(1)

```
namespace pmr {
    template< class T >
    using vector = std::vector<T, std::pmr::polymorphic_allocator<T>>;
}
```
(2) (since C++17)

1) `std::vector` is a sequence container that encapsulates dynamic size arrays.

2) `std::pmr::vector` is an alias template that uses a polymorphic allocator.

The elements are stored contiguously, which means that elements can be accessed not only through iterators, but also using offsets to regular pointers to elements. This means that a pointer to an element of a vector may be passed to any function that expects a pointer to an element of an array.

The storage of the vector is handled automatically, being expanded as needed. Vectors usually occupy more space than static arrays, because more memory is allocated to handle future growth. This way a vector does not need to reallocate each time an element is inserted, but only when the additional memory is exhausted. The total amount of allocated memory can be queried using `capacity()` function. Extra memory can be returned to the system via a call to `shrink_to_fit()`[1].

Reallocations are usually costly operations in terms of performance. The `reserve()` function can be used to eliminate reallocations if the number of elements is known beforehand.

The complexity (efficiency) of common operations on vectors is as follows:

- Random access - constant $\mathcal{O}(1)$.
- Insertion or removal of elements at the end - amortized constant $\mathcal{O}(1)$.
- Insertion or removal of elements - linear in the distance to the end of the vector $\mathcal{O}(n)$.

`std::vector` (for T other than `bool`) meets the requirements of *Container*, *AllocatorAwareContainer* (since C++11), *SequenceContainer*, *ContiguousContainer* (since C++17) and *ReversibleContainer*.

Member functions of `std::vector` are `constexpr`: it is possible to create and use `std::vector` objects in the evaluation of a constant expression.
However, `std::vector` objects generally cannot be `constexpr`, because any dynamically allocated storage must be released in the same evaluation of constant expression.
(since C++20)

1. ↑ In libstdc++, `shrink_to_fit()` is not available in C++98 mode.

Template parameters

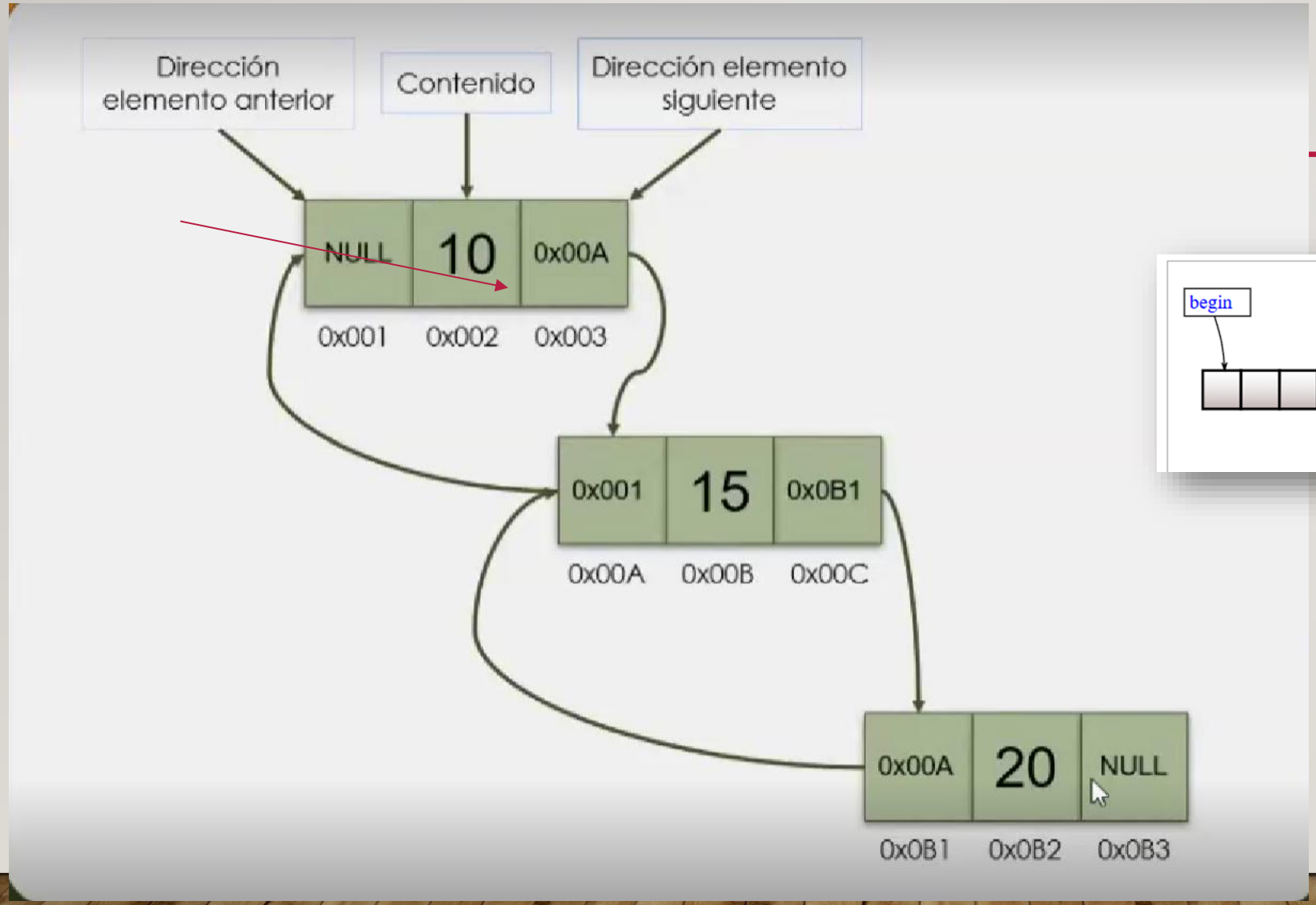
T — The type of the elements

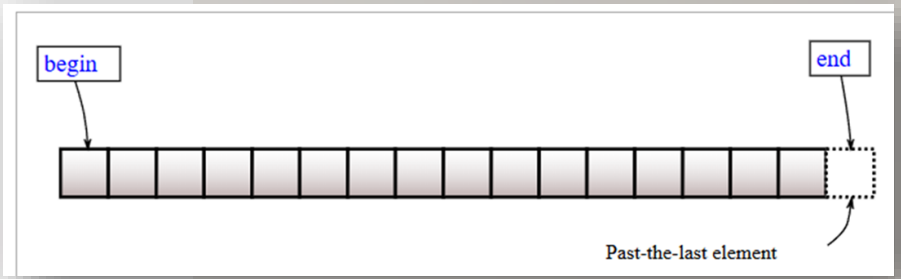
LIST

- Una lista es una estructura doblemente enlazada, debido a que cada elemento de la lista contiene la referencia del dato siguiente y el dato anterior, a través de apuntadores.
- Al igual que un vector o un arreglo, almacena una secuencia de elementos.
- Los elementos de la lista no necesariamente son contiguos en memoria.
- Se debe usar una lista cuando se desee, insertar o remover muchos elementos, pero no se requiera una búsqueda rápida.

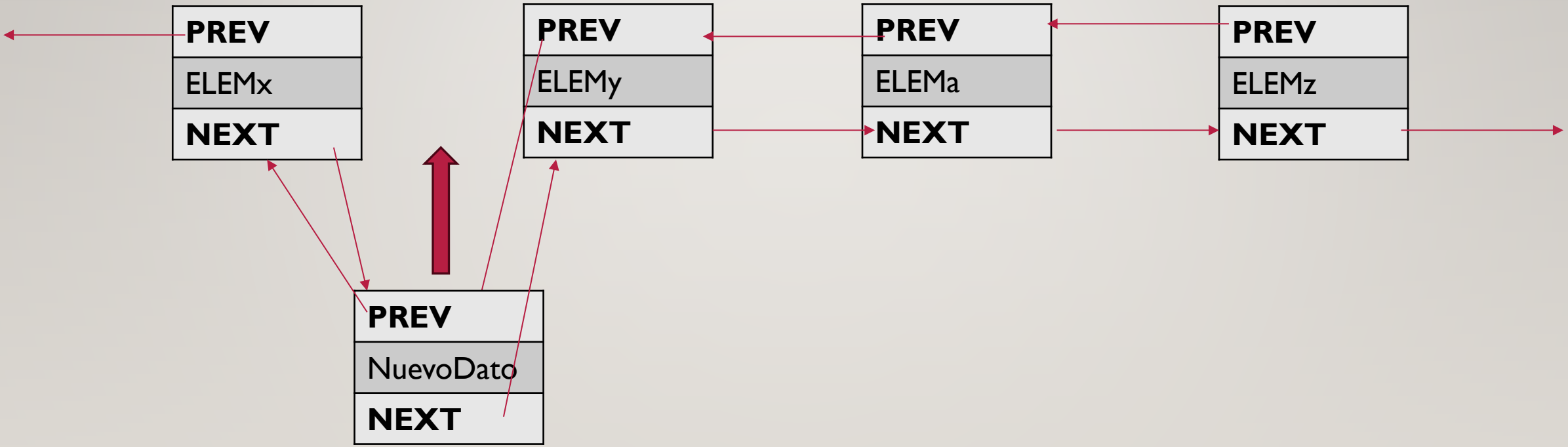
ALMACENAMIENTO EN LISTAS

ista





EJEMPLO DE INSERCIÓN



LIST vs ARRAY vs VECTOR

LIST	ARRAY	VECTOR
A list is a sequential container to store elements, it is not index based.	Array stores a fixed-size sequential collection of elements of the same type and it is index based.	-
-	As array is fixed size, once initialized can't be resized.	Vector is dynamic in nature so, size increases with insertion of elements.
-	Array is a memory efficient data structure.	Vector occupies more memory than _____
-	Array access elements in constant time, since elements are arranged in contiguous memory.	Vector takes more time in accessing elements than _____
-	-	Vectors can be automatically moved in memory if _____

FIN

ANEXO: ITERADORES

ITERADORES

- Cada contenedor proporciona un iterador, el cual es un puntero que sabe como iterar secuencialmente sobre los elementos de un contenedor específico.
- Todos los iteradores son construibles, asignables y destruibles.
- Tienen la ventaja de ser genéricos, trabajando sobre contenedores de clases definidas por el programador.
- Los iteradores en C++ no necesitan ser liberados explícitamente. **Nota**: esto no significa que la memoria de los objetos internos de un contenedor no debe liberarse.

APUNTADORES vs ITERADORES

Apuntadores	ITERADORES
Variable que almacena una dirección de memoria	Es un objeto que permite gestionar de manera eficiente operaciones vinculadas al acceso y recorrido de los elementos de un contenedor. Constituye la generalización de un apuntador.
Se pueden utilizar operadores aritméticos como --, ++, [ ], +, - .	Está restringido en los operadores a usar, respecto al tipo de contenedor
Un apuntador de tipo T* puede apuntar a cualquier tipo de dato.	Se deben declarar específicamente para el tipo de contenedor a usar.